C Bitwise Operators Cheatsheet

Quick reference for firmware / BIOS register programming

1. The operators

Operator	Name	Works on	Compound form
&	AND	two values, bit by bit	&=
|	OR	two values, bit by bit	|=
^	XOR	two values, bit by bit	^=
~	NOT (complement)	one value	none
<<	Shift left	value, count	<<=
>>	Shift right	value, count	>>=

2. Truth tables (the rules)

A	B	A & B	A | B	A ^ B	~A
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Operator	Rule in one line
AND &	Result is 1 only if **both** bits are 1
OR |	Result is 1 if **at least one** bit is 1
XOR ^	Result is 1 if the bits are **different**
NOT ~	**Flips** every bit

3. What each one is used for (mask M)

Operator	Used to	With mask M
&	Clear, test, extract bits	X & M keeps only the bits where M is 1
|	Set bits	X | M forces to 1 the bits where M is 1
^	Toggle bits, compare, swap	X ^ M flips the bits where M is 1
~	Build an inverted mask	X & ~M clears the bits where M is 1

Per-bit identities (x is any bit)

Expression	Result	Meaning
`x & 0`	`0`	AND with 0 clears
`x & 1`	`x`	AND with 1 keeps
`x | 0`	`x`	OR with 0 keeps
`x | 1`	`1`	OR with 1 sets
`x ^ 0`	`x`	XOR with 0 keeps
`x ^ 1`	`~x`	XOR with 1 flips
`x ^ x`	`0`	XOR with itself gives 0

4. Worked example (8-bit)

```
A = 1100 1010 (0xCA)
B = 1010 0110 (0xA6)

A & B = 1000 0010 (0x82)
A | B = 1110 1110 (0xEE)
A ^ B = 0110 1100 (0x6C)
~A    = 0011 0101 (0x35)
```

5. Shifts

Operator	Effect	Fill bits	Math meaning
`X << n`	Moves bits left by n	0s enter on the right	multiply by 2^n
`X >> n`	Moves bits right by n	0s on the left (unsigned); sign-dependent for signed	divide by 2^n

```
0000 0110 (6)   << 2  =  0001 1000 (24)
1001 0000 (144) >> 4  =  0000 1001 (9)
```

`1 << n` creates a mask with only bit n set. This is how `BIT0`, `BIT1`... are defined: `#define BIT3 (1 << 3)` equals `0x08`.

6. Firmware idioms

Task	Code	xHCI example
Set bit	`Reg |= BITn;`	`Cmd |= XHCI_CMD_HCRST;`
Clear bit	`Reg &= ~BITn;`	`Cmd &= ~XHCI_CMD_RS;`
Toggle bit	`Reg ^= BITn;`	`flip a flag`
Test bit is set	`if (Reg & BITn)`	`if (Sts & XHCI_STS_HCH)`
Test bit is clear	`if (!(Reg & BITn))`	`the earlier ASSERT line`
Extract a field	`(Reg >> shift) & mask`	`(Reg >> 8) & 0xFF`
Insert a field	`Reg = (Reg & ~(mask << shift)) | (Val << shift);`	`write a field, keep the rest`
Power of 2 check	`(X & (X - 1)) == 0`	`alignment checks`
Align up	`(X + A - 1) & ~(A - 1)`	`A must be a power of 2`

7. Compound operators

Write	Same as
`A &= B`	`A = A & B`
`A |= B`	`A = A | B`
`A ^= B`	`A = A ^ B`
`A <<= n`	`A = A << n`
`A >>= n`	`A = A >> n`

8. Common mistakes

Mistake	Problem	Correct
& vs &&, | vs ||	&& and || are **logical** (true/false); & and | work on bits	Use & and | for registers

Mistake	Problem	Correct
if (Reg & BIT3 == BIT3)	== binds tighter than &, so it evaluates wrong	if ((Reg & BIT3) == BIT3), always add parentheses
Reg &= BITn to clear	That keeps only that bit	Reg &= ~BITn
+ instead of | to set	Breaks if the bit is already set	Use |=
~ on small types	~ promotes UINT8 to int, so upper bits are set; comparisons may fail	Cast: (UINT8)~Data8
Shifting signed values	>> on negatives is implementation-defined; << into sign bit is undefined	Use unsigned types
Shifting too far	X << 32 on a 32-bit value is undefined	Keep n below the type width
RW1C bits	Reg |= X may accidentally clear write-1-to-clear status bits	Read the spec; write 1 only to the bit you intend to clear

9. Quick memory aid

Operator	Think	Use to
AND	both must be 1	keep / clear / test
OR	any is 1	set
XOR	different	flip
NOT	invert	build clear-masks
Shift	move bits	build masks, extract fields